

FERAL GLOSS EMPIRE

Lineage Simulation Build — Full Session Compile

This document is the complete record of the architectural session that produced the FGE Lineage Simulation engine. It captures every concept, decision, counterweight position, key insight, working artifact, and the reasoning behind each. It is a living document: the Trait Matrix remains a reconstruction awaiting correction from the lost run. That correction is the founding delta. The build is not finished; the build is seeded.

SESSION ARCHITECTURE IN ONE SENTENCE

A slow, append-only, attention-gated lineage engine where the deliverable is the cinematic photograph of a dragon — not the simulation itself — and where the user steers a little, documents a lot, and cannot undo what has grown.

DOCUMENT MAP

Chapter I — The Grok Report — What It Got Right and What It Was Selling

Chapter II — The Lineage Object — The Idea That Hasn't Been Made

Chapter III — Spec-Then-Test — Why and the Thin Spec

Chapter IV — The Sandbox Vision — SimCity Is Not the Point

Chapter V — The Safari Model — Voyeur, Not Manager

Chapter VI — The Lost Run — 25 Cycles and the Lawmaster

Chapter VII — The Trait Matrix — 15 Fixed Traits, the Measurement Spine

Chapter VIII — The Four Documents — Pattern Named, Traps Identified

Chapter IX — The Five Vocabularies — One Engine, Five Costumes

Chapter X — The Founding Delta — Reconstruction as Input to Seeding

Chapter XI — ARTIFACT: Lineage Object Thin Spec v0.1

Chapter XII — ARTIFACT: Injection Contract v1

Chapter XIII — ARTIFACT: Trait Matrix v1

Chapter XIV — ARTIFACT: Grok Lineage Directive v1

Chapter XV — What Ships Next — The Honest Next Move

The Grok Report — What It Got Right and What It Was Selling

The session opened with a Grok Business Maven Mode report on the DragonOS v2.1 expansion. The report correctly identified DragonOS as infrastructure, not content, and proposed a three-tier product ladder (\$79–\$1,500+). It coined the positioning: *market the genome language, not the dragons*.

KEY INSIGHT: The infrastructure-not-content framing is correct and is the same instinct behind Seed Theory. The moat is the genome language and lineage, not any single asset.

What the report got right:

The verb ladder Watch → Guide → Rewrite. The tiering logic (free embryo → \$79–149 genome → \$399–699 framework → high-roller revenue share). Its own honesty: it flagged Heartfield Dynamics, Observer Infection, and cross-domain reusability as unproven.

Where it would have hurt:

COUNTERWEIGHT: The three unproven gaps are load-bearing for the pricing. Strip Heartfield, Observer Infection, and cross-domain ports — none implemented — and Tier 2/3 lose their entire value proposition. What remains that is actually production-ready is the Genome Matrix + Invocation Stack + Prompt Export Compiler. That is a Tier 1 product. The report was charging \$699 for vapor wrapped around a \$99 deliverable.

COUNTERWEIGHT: Revenue-share at Tier 3 with no contract substrate. \$1,500+ or revenue share means ongoing deliverables on capability that does not exist yet, to the highest-paying customers. That is the worst possible group to under-deliver to.

The counter-move established: demote to a sequencing problem. Ship only what is production-ready as a single Tier 1 SKU. Hold Tier 2/3 as roadmap, not inventory. Let the first real sale validate the infrastructure story before building the contract obligations that come with selling it.

One real attribution first. Then DragonOS earns its OS.

The Lineage Object — The Idea That Hasn't Been Made

The Chairman's brainstorming produced a new idea: a collectible where lore accrues structurally, through provenance and lineage, without anyone narrating it. No user risk. No authoring. The held object deepens as the lineage grows beneath it.

Most collectibles are static — you own a card, it sits there, its meaning is fixed at mint. Lore-heavy worlds are opposite — high-risk, because meaning lives in a narrative someone has to read, defend, and not contradict. What you landed on is the third thing: the held object accrues lore from the structure around it, without the holder authoring anything.

The core principle:

The card does not contain the story. The card's **position in the lineage** is the story. Ownership is the read action. That is the unmade object.

Four vertebrae of the spine:

Vertebra 1 — The object carries identity, not narrative. Each collectible binds to five invariant fields: sbid, material fingerprint, frequency cluster, core conflict, signal. No prose. The five fields ARE the lore, encoded.

Vertebra 2 — Lore accrues by lineage, not by telling. When a new genome derives from an owned one, the held object gains depth automatically. The holder did nothing. The structure deepened the object. That is lore building without speaking it.

Vertebra 3 — The detective layer prices, it does not create. Canon evaluation reads the value the lineage already deposited. It does not make the value. The canon detective assesses worth on its own.

Vertebra 4 — Growth has a gate, or it is not a spine — it is a pile. A derivation only deepens an ancestor's value if it passes canon lock. Junk descendants do not dilute the line because they never enter it.

KEY INSIGHT: The origin object (SB_330 / Raven / Obsidian) is the most load-bearing object that can exist in the system — but only once descent accrues beneath it. Origin without descendants is potential, not value. This encoding means value is earned, not granted at mint.

COUNTERWEIGHT: Owning + accruing value + lineage depth starts to look like a speculative asset. Keep accrual expressive/canonical (depth, rarity, position) and resist framing it as financial return. That keeps the no-user-risk promise true in the way that actually protects you.

Spec-Then-Test — Why and the Thin Spec

The decision to spec before testing was the exception to the standing north star (test before doctrine, ship the card before you believe the system). The reason:

If we hand-trace Raven's lineage right now, we would be measuring accrual against rules we have not written down — so any result is unfalsifiable. You would see depth accruing and not know if it is the structure working or you wanting it to work.

The thin spec: minimum to make the test falsifiable. Three elements, no more:

Element 1 — The Accrual Rule (formula):

```
VALUE(obj) = (w_d * DEPTH) + (w_o * ORIGIN * DEPTH) + (w_r * RARITY)
DEPTH(obj) = number of LOCKED descendants below obj
ORIGIN(obj) = 1 / (hops from obj to line origin + 1)
RARITY      = 1 / (count of LOCKED objects sharing frequency_cluster)
Weights: w_d=1.0, w_o=2.0, w_r=0.5
```

ORIGIN multiplies DEPTH (not adds) because an origin with no descendants is potential, not value. Value is earned only when the line grows beneath it.

Element 2 — The Lock Gate (one law):

A derivation deepens an ancestor's VALUE if and only if the derivation passes canon lock (compile → validate → lock). Unlocked derivations are invisible to the tree.

Element 3 — The Detective Output Contract:

```
{
  "sbid": "SB_330",
  "value": 0.0,
  "components": { "depth": 0, "origin": 1.0, "rarity": 0.0 },
  "position": { "origin_distance": 0, "locked_descendants": 0 },
  "grade": "ANCHOR | DEEP | DERIVED | LEAF",
  "narrative_emitted": false
}
```

narrative_emitted MUST be false. If the evaluator produces prose to justify worth, the contract is violated and 'lore without speaking it' has failed.

KEY INSIGHT: Pass condition for the test: VALUE rises for an object whose owner did nothing — purely because locked descendants entered the line. If VALUE only moves when the holder acts, the spine is dead and 'owning = reading' is false.

The Sandbox Vision — SimCity Is Not the Point

The Chairman introduced the cultivation dimension: users tend a living lineage that grows into dragons, cities, civilizations. The reframe:

KEY INSIGHT: The simulation is not the product. The images are the product. The simulation exists to make the images easy to produce, consistent, and worth caring about.

The two product lanes established:

The Object — you own, lore accrues, no agency. Lowest risk, lowest engagement, closest to ship.

The Sandbox — you tend, growth responds, low-but-nonzero risk. Higher engagement, higher build cost, render layer required.

COUNTERWEIGHT: The Sandbox is the destination; the Object is the proof. The Object validates that lineage-accrual reads as value with no engine to build. If people pay to hold an accruing lineage, the core loop is proven before spending a year building the tending interface.

The honest version of 'grows into civilizations':

The user nurtures a lineage, and the render layer expresses that lineage as a poster, card, city-tile, dragon. The growth is real (the tree deepens); the 'civilization' is a **deterministic render of the tree's shape**, not a simulated society. Ship the render of real structure. Do not promise the simulation of fake emergence.

COUNTERWEIGHT: SimCity lives or dies on whether the user's small action produces a legible, surprising, non-narrated consequence. Your lock gate is perfect for this: an injection either passes lock and becomes lineage, or it does not and the seed ignores it. Death-by-neglect is just absence of accrual. That is elegant and it is already in the spec.

The Safari Model — Voyeur, Not Manager

The Chairman clarified the genre precisely: like the old trophy-hunter games. You walk around the map, shoot (photograph), collect trophies. You cannot build cities, cannot take cars, cannot manage anything. You witness and document.

The voyeuristic nature I want this simulation to have is all of these backend growth occurrences happening, but the user can't just delete a dragon. Something else that's growing — you can't create global shift. You have a very limited option. Once it's grown, it's grown. You can start a new game but you can't change evolution as it's occurring — that's locked unless you stop it.

The five core laws that followed:

LAW_TICK: Evolution advances ONLY on the tick. The user cannot rush, skip, or rewind. Growth is the engine's, not the user's.

LAW_APPEND_ONLY: Nothing that has grown can be deleted, edited, or reverted. A dragon, once locked into the lineage, is permanent.

LAW_STEER_NOT_AUTHOR: The user injects directional pressure BEFORE a tick resolves. They influence what MIGHT grow next. They never author the result directly.

LAW_NEW_GAME_ONLY: The only way to undo is to start a new lineage. An existing lineage is read-only history.

LAW_DELIVERABLE_IS_THE_IMAGE: The product is the render (photo/video) of a state. The simulation exists to produce photographable states, not to be managed.

The disclaimer shown at simulation start:

This is a slow simulation. Your dragons grow on their own, on a timeline you cannot speed up. You may gently steer what grows next — but once something has grown, it is permanent. You cannot delete it, edit it, or change its past. You document; you do not control. To begin again, start a new lineage.

KEY INSIGHT: The tick can HOLD — most ticks, nothing happens. The world is just there, growing slowly, and the user walks through and photographs it. The slowness is not a limitation; it is the product.

The Lost Run — 25 Cycles and the Lawmaster

The most important revelation of the session: the Chairman had already run this for ~25 cycles in a prior Grok account, with a tree (not an embryo) as the origin object. What emerged:

What came out of it was a Lawmaster and new products and buildings, a society that would gather around the tree, which was what it was. It was not an embryo — a tree — and it was amazing and I want to reproduce that.

What made it work:

The fixed-matrix discipline. 15 traits, every millimeter, every angle, every cycle, logged whether or not anything interesting happened. Dimensional consistency across all 25 cycles made the trajectory derivable. Grok could draw the clear path because the data was consistent. Most people never do this. You did, and that is why yours produced a Lawmaster.

The preamble-as-ledger. All deltas in the preamble meant every cycle re-grounded on full accumulated history before growing. No cycle started from scratch. That is append-only memory enforced through prompt structure.

Attention as the growth mechanism. It won't track what doesn't have attention. The tree grew where you looked. That was not philosophy; it was the operating constraint of the system.

COUNTERWEIGHT: The genius and the fatal flaw were the same architecture. The preamble was the entire memory, and the preamble lived in someone else's account. The idea is only as durable as the ledger it lives in. Last time the ledger was somebody else's account.

The single upgrade from the lost run:

KEY INSIGHT: The preamble becomes a file you hold. Same function — every cycle re-reads full accumulated state — but the state lives in your JSON, not a chat. The tree can never be lost in an account again. Grok narrates; your file remembers.

The tree as origin-node insight:

An embryo is a thing that becomes one organism. A tree is a thing a society gathers around — the origin point that everything else orients toward. The Lawmaster, the buildings, the locals all arranged themselves in relation to it. That is your origin node (Emberling / Raven / SB_330) doing exactly what the thin spec said: accruing load as everything derives toward it. The lost run proved the accrual mechanic works before the spec was written.

The Trait Matrix — 15 Fixed Traits, the Measurement Spine

Reconstructed from the Chairman's description of the lost run. Fifteen traits, fixed, measured every cycle without exception. The measurement law:

Every trait logged every cycle. A trait that did not change is logged as delta 0 — never omitted. Omission breaks the trajectory.

The five groups and their traits:

EMERGENCE (T01–T02)

T01 emergence_height (mm) — how far the subject has risen above ground. T02 broke_ground (boolean) — has it emerged at all.

STRUCTURE (T03–T06)

T03 trunk_diameter (mm). T04 branch_count (count). T05 branch_length_max (mm). T06 branch_angle_dominant (degrees from vertical).

FOLIAGE (T07–T09)

T07 leaf_count (count). T08 leaf_color_dominant (hex/name). T09 leaf_color_variance (how many distinct colors — mono vs varied).

TRAJECTORY (T10–T12)

T10 growth_trajectory_vector (net direction: upward, lateral, canopy-spread). T11 growth_angle_overall (lean from vertical, degrees). T12 growth_velocity (total mass-change rate this cycle vs last).

CHARACTER (T13–T15)

T13 symmetry_index (0.0–1.0; asymmetry signals character/age). T14 anomaly_marker (any pronounced deviation this cycle — fork, scar, unexpected sport — the seed of narrative). T15 categorization (DERIVED — the subject's developmental stage read from the other 14: sprout → sapling → established → elder → monument).

KEY INSIGHT: T14 and T15 are where the Lawmaster came from. A society does not gather around uniform growth — it gathers around deviation that gets read as meaning. T14 captures the deviation; T15 reads the accumulated traits into a stage the locals can recognize. Those two are the bridge from 'a tree grew' to 'a society formed.'

The two governing laws:

The attention law: A subject only grows on a cycle where it received attention (attention_flag = true). Unattended subjects are logged but not advanced. The tree grows where you look. This is now mechanical, not philosophy.

The preamble law: Every cycle begins by re-reading ALL prior records. Growth is computed against full accumulated history, never against last cycle alone. The ledger file IS the preamble — and it is yours, not an account's.

COUNTERWEIGHT: Do not expand the matrix while rebuilding it. The lost run worked at 15. Fifteen traits tracked perfectly every cycle beat fifty traits tracked sometimes. Rebuild at 15. Earn more only after 25 fresh cycles prove the substrate holds.

The Four Documents — Pattern Named, Traps Identified

Four documents from other AI systems were reviewed during the session. Each was assessed for what was real and what was trap.

The pattern across all four:

COUNTERWEIGHT: Every assistant you show this to generates the catalog instead of the inventory. Because inventing 30 sexy item names with prices is free and fast, and telling you 'I cannot sort these because you have not given me the real ones' is unsatisfying. They optimize for the satisfying output.

Document 1 — The 30 Injectors / Marketing Framework:

What was right: injectors-not-extras reframe (correct). Verb ladder Watch → Guide → Rewrite (permanent keeper). Object-safe vs Sandbox-only split (actionable now).

COUNTERWEIGHT: Selling 'steer reality drift inside a living lineage' before the injection mechanic exists. Every category (Civilization Bias Modules, Archetype Pressure Keys) promised emergent behavior the engine cannot yet deliver.

Document 2 — Canon Cultivation OS / n+1–n+3 Dashboard:

What was right: upgrades = increasing levels of awareness (correct). The Gardener's Table (3 things to do today — the best single panel in any document). n+4 hesitation — independently rediscovered the ceiling.

COUNTERWEIGHT: Forty metrics, zero formulas. Entropy Score: Low. Future Leverage: 9.8. Every field is a dashboard render of numbers nothing currently computes. This is the dashboard-before-the-data trap.

Document 3 — The Fictional Catalog (\$5–\$10,000+):

COUNTERWEIGHT: 'The Black Pearl — Unknown function — Future leverage: 10/10.' Selling an item whose function is explicitly unknown at auction tier. None of these were real assets. They were names a language model generated to fill a rarity ladder.

Document 4 — CharacterOS (six specs, implementation pending):

The most important document because it is the Chairman's own work. The status line at the bottom:

Architecture Complete, Implementation Pending.

COUNTERWEIGHT: Re-specifying is satisfying — each pass feels like progress, the diagrams get prettier, the brand language sharpens. But six specs of the same system is not six times the progress. It is the same rung, climbed six times.

KEY INSIGHT: CharacterOS is the factory that produces the objects the Lineage spine accrues value to. Phase 5 (Canon Lock and Transport) IS the lock gate from the thin spec. They are not two projects. They are the factory, the thing it makes, and how the thing accrues worth.

The Five Vocabularies — One Engine, Five Costumes

A critical convergence observation from the session:

KEY INSIGHT: DragonOS, CharacterOS, the Lineage Object, the Cultivation OS, the 30 protocols — these are one engine wearing five costumes. The risk is not that any one is wrong. It is re-specifying the engine in a new costume instead of shipping one artifact the engine already produces.

The proof of convergence:

Raven (SB_330) is the origin node in the collectible vocabulary. Emberling (DRG-0001, generation 0, the calibration flame) is the origin node in the dragon vocabulary. They are the same role in the same engine. Tracing one proves the other. There are not two pilots to run. There is one.

The DragonOS document's own honest question (asked and then fled from):

The vagueness you feel is due to a specific gap. You have defined structure, taxonomy, materials, evolution, rendering, validation, identity persistence — but not agency. The question is: what is generated? When DragonOS finishes, what exists? Is it an image, a character dossier, a persistent organism, an actor capable of making decisions? These are radically different destinations.

The answer, given by the Chairman directly:

The engine is ultimately for cinematic beauty of dragons and dragon lore, to produce, to log, to build their own sort of dragon architecture — if they want something that grows on its own, uses lineage, builds families. The simulation breathes life into something that the user may already be creating, or in the case that they only want to see dragons and only want to take pictures of dragons, they have the liberty to breed whatever they want.

KEY INSIGHT: The simulation is scaffolding for image generation. The lineage is what makes the second image related to the first instead of a stranger. The 'growth' only has to go far enough to generate the next photographable state. That is not a multi-year engine. That is reachable.

The Founding Delta — Reconstruction as Input to Seeding

The Chairman's precise sequencing of the trait matrix rebuild:

Your output is the input to the seeding. Once I get the traits and bring them back we will have a delta which is high value.

Why this is the correct sequencing, stated as a system principle:

The reconstruction is a **prediction** of the matrix. The Chairman's memory is the **ground truth**. The delta between them is the first thing the system learns — and per the preamble law, it gets logged, not discarded. The reconstruction was never meant to be right; it was meant to be a measurable starting position so that when truth arrives, the correction is captured as the opening entry in the ledger rather than silently overwritten.

KEY INSIGHT: You are treating the reconstruction the way the engine treats a cycle: a state to be measured against, with the delta as the value. Most people throw away the draft when the real one arrives. You are keeping the difference. That is the whole discipline.

What happens when the real 15 return:

Step 1 — Compute the founding delta. Reconstruction vs. truth, logged as cycle 0's opening entry, the first high-value record in the ledger.

Step 2 — Lock the corrected matrix. The real 15, fixed, the permanent spine.

Step 3 — Seed cycle 0. The tree breaks ground with the true traits at starting values. Emberling / the tree / the origin node enters the ledger.

Step 4 — Bundle everything. PDF + ZIP, clean, no draft floating beside it. One portable document the substrate cannot lose.

Your output becomes the input. The delta becomes the first thing the tree remembers.

ARTIFACT: Lineage Object Thin Spec v0.1

The accrual spine. VALUE formula, lock gate, detective output contract. Written to make the Raven trace falsifiable. Three elements, no more. Pass condition: VALUE rises on its own, narrative_emitted stays false.

```
# FGE Lineage Object – Thin Spec v0.1
```

```
**Status:** PRE-DOCTRINE / TEST-ENABLING ONLY
```

```
**Purpose:** The minimum set of rules required to make the Raven SB_330 hand-trace *falsifiable*.
```

```
**Not in scope (deferred until test holds):** five-field binding ceremony, L0-L3 reconciliation, regulatory/trust framing, G
```

```
**One question this spec must let us answer:** *Does a held object deepen, mechanically, without anyone narrating it?*
```

```
---
```

Element 1 – The Accrual Rule (formula)

An object's worth is a function of its **position in the append-only lineage tree**, not its narrative and not its mint date

```
```
```

```
DEPTH(obj) = number of LOCKED descendants below obj in the tree
```

```
ORIGIN(obj) = 1 / (hops from obj to its line's origin + 1) # origin itself = 1.0
```

```
RARITY(cluster) = 1 / (count of LOCKED objects sharing obj.frequency_cluster)
```

```
VALUE(obj) = (w_d · DEPTH) + (w_o · ORIGIN · DEPTH) + (w_r · RARITY)
```

```
```
```

Default weights for the test: `w\_d = 1.0`, `w\_o = 2.0`, `w\_r = 0.5`.

Why ORIGIN multiplies DEPTH and doesn't just add: an origin object with no descendants is *potential*, not *value*. Value

Falsification target: if VALUE rises for an object whose owner did nothing – purely because *other* locked descendants er

```
---
```

Element 2 – The Lock Gate (one law)

> **LAW:** A derivation deepens an ancestor's VALUE **if and only if** the derivation passes canon lock (compile → validate

This is the single line that separates a spine from a pile. Without it, growth is unfalsifiable – everything accrues, so not

Test consequence: during the Raven trace, any descendant not yet locked is scored as **not present**. We measure VALUE a

```
---
```

Element 3 – The Detective Output Contract

The canon evaluator (Atelier Brain) **prices**; it does not create. Given an object's sbid and its tree position, it returns

```
```json
```

```
{
 "sbid": "SB_330",
 "value": 0.0,
 "components": { "depth": 0, "origin": 1.0, "rarity": 0.0 },
 "position": { "origin_distance": 0, "locked_descendants": 0 },
 "grade": "ANCHOR | DEEP | DERIVED | LEAF",
 "narrative_emitted": false
}
```

```
```
```

- `narrative\_emitted` MUST be `false`. If the evaluator ever produces prose to justify worth, the contract is violated and "

- `grade` is a *read* of position, never an authored judgment:

- **ANCHOR** – origin_distance 0

- **DEEP** – has locked descendants

- **DERIVED** – has a locked ancestor and locked descendants

- **LEAF** – locked, no descendants yet

- This JSON is the thing you stare at during the test. The pass condition is subjective but bounded: *does this read as accru*

The Test Protocol (what v0.1 exists to enable)

1. Place **Raven / SB_330 / Obsidian** as the line origin.
2. Enumerate its **real, currently-locked** descendants – by hand, lock-gated.
3. Compute VALUE for Raven and for 2-3 mid-tree objects using Element 1.
4. Emit the Element 3 JSON for each.
5. Judge the one question: **did Raven's VALUE rise solely from descendants accruing beneath it, with `narrative_emitted: fa**

Pass → earn `FGE-LINEAGE-OBJECT-001` (full doctrine).

Fail → we changed three rules, not a doctrine. North star intact.

ARTIFACT: Injection Contract v1

The safari-model growth mechanic. Five core laws, three closed injections (BREED / AGE_ADVANCE / TRAIT_PRESSURE), node schema with deterministic map coordinates, and the deliverables-per-node contract. The map is a byproduct of growth, not a thing you draw.

```
{
  "contract_id": "FGE-INJECTION-CONTRACT-v1",
  "status": "TEST-ENABLING / PRE-DOCTRINE",
  "depends_on": "FGE-LINEAGE-OBJECT-THIN-SPEC-v0.1",
  "genre": "safari_documentation",
  "one_line": "The world grows slowly on a mandatory tick. The user steers a little, documents a lot, and cannot undo what has happened.",

  "core_laws": {
    "LAW_TICK": "Evolution advances ONLY on the tick. The user cannot rush, skip, or rewind the tick. Growth is the engine's heartbeat.",
    "LAW_APPEND_ONLY": "Nothing that has grown can be deleted, edited, or reverted. A dragon, once locked into the lineage, is permanent.",
    "LAW_STEER_NOT_AUTHOR": "The user injects directional pressure BEFORE a tick resolves. They influence what MIGHT grow next, but do not control it.",
    "LAW_NEW_GAME_ONLY": "The only way to 'undo' is to start a new lineage (new game). An existing lineage is read-only history.",
    "LAW_DELIVERABLE_IS_THE_IMAGE": "The product is the render (photo/video) of a state. The simulation exists to PRODUCE photographs.",
  },

  "disclaimer_shown_at_start": "This is a slow simulation. Your dragons grow on their own, on a timeline you cannot speed up or slow down.",

  "tick": {
    "type": "mandatory_slow",
    "user_can_rush": false,
    "user_can_skip": false,
    "user_can_rewind": false,
    "resolution": "On each tick, at most ONE pending injection resolves into a new locked node, OR the world holds (no growth).",
    "note": "Tick cadence is a tuning value, intentionally long. The voyeuristic feel depends on the user NOT being able to speed up.",
  },

  "node_schema": {
    "comment": "Every dragon is a node in the lineage graph. This is the unit the renderer photographs and the unit the map plots.",
    "node_id": "string (e.g. DRG-0001)",
    "lineage_id": "string (which game/run this belongs to)",
    "parents": "array[node_id] (0 for seed, 1 for age/trait, 2 for breed)",
    "children": "array[node_id]",
    "generation": "integer (seed = 0)",
    "born_on_tick": "integer",
    "locked": "boolean (only locked nodes count for lineage, render, and map)",
    "genome_ref": "string (points to the canon genome / invariant kernel)",
    "visual_anchor_ref": "string (the Anchor Vector that keeps identity recognizable across renders)",
    "map_position": {
      "comment": "MAPPABILITY. Every node gets deterministic coordinates so the lineage IS a map with no extra work.",
      "x": "float = generation (time axis, left to right)",
      "y": "float = lineage_lane (which bloodline branch, top to bottom)",
      "derivation": "Coordinates are computed from graph position, never hand-placed. A new node's (x,y) is fixed the instant it is born.",
    },
    "render_state": {
      "ready": "boolean",
      "ceremony": "enum [birth, elder_rite, territorial_clash, call_to_action, dormant] (the FRAMING for the photo, not a state)",
      "render_ref": "string (points to compositor output once photographed)"
    }
  },

  "injections": {
    "comment": "The CLOSED v1 set. Exactly three moves. Each is lock-gated. A wide menu is v3; v1 stays narrow so every result is high quality.",
    "BREED": {
      "inputs": ["parent_a: node_id (must be locked)", "parent_b: node_id (must be locked)"],
      "produces": "one descendant node carrying both parent lineages",
      "lock_gate": "Descendant must pass canon lock (genome compatible, no illegal cross-class traits). If it fails, NO node is produced.",
      "map_effect": "New node placed at generation = max(parents)+1, on a lane between the two parent lanes.",
      "render_default_ceremony": "birth"
    },
    "AGE_ADVANCE": {
```

```

    "inputs": ["subject: node_id (must be locked)", "direction: enum [juvenile, elder]"],
    "produces": "a new node, SAME identity, advanced life stage \u2014 NOT a replacement (append-only; the prior stage persists)",
    "lock_gate": "Anchor Vector must be preserved (same recognizable dragon). If identity drift exceeds tolerance, reject a new node.",
    "map_effect": "New node placed adjacent on the same lineage lane, one generation step along the time axis.",
    "render_default_ceremony": "elder_rite"
  },
  "TRAIT_PRESSURE": {
    "inputs": ["subject: node_id (must be locked)", "pressure: enum [volcanic, glacial, abyssal]"],
    "produces": "a new adapted node \u2014 same dragon, environment-shifted visual state (your Evolution Engine logic: pressure)",
    "lock_gate": "Adaptation must be biologically coherent for the genome (no aquatic lungs + volcanic respiration). Reject a new node.",
    "map_effect": "New node branches off the subject's lane as a sub-branch.",
    "render_default_ceremony": "call_to_action"
  }
},
"what_the_user_CANNOT_do": [
  "delete any node",
  "edit a locked node's genome, anatomy, or history",
  "rewind or replay a past tick",
  "fast-forward the tick",
  "directly author a result (they steer pre-tick; the engine resolves)",
  "force growth (the world may hold a tick with no change)",
  "manage resources, traffic, conflict outcomes, or 'win/lose' states \u2014 there are none"
],
"what_the_user_CAN_do": [
  "queue ONE injection before a tick (breed / age / trait-pressure)",
  "photograph any locked node, in any available ceremony framing (image or video)",
  "collect renders into their lineage gallery (the trophy wall)",
  "let the world simply grow and watch / document",
  "start a NEW lineage at any time (the only reset)"
],
"deliverables_per_node": {
  "image": "compositor render of the node in chosen ceremony framing",
  "video": "short loop of the node (the A/B video spec you already build)",
  "map": "the lineage graph, auto-plotted from node map_position values \u2014 the user's 'territory'"
},
"handoff": {
  "comment": "How this plugs into what you already have.",
  "to_lineage_spine": "A locked node here = a locked descendant in the thin spec. DEPTH/ORIGIN/RARITY (and therefore VALUE) is preserved.",
  "to_compositor": "render_state.render_ref points at fge_compositor.py output. The contract decides WHAT to photograph; the engine decides HOW.",
  "to_canon": "lock_gate in every injection IS the compile->validate->lock lifecycle. No new lock machinery needed."
},
"map_is_free": "Because map_position is computed from generation (x) and lineage lane (y) at lock time, the lineage graph is free to evolve."
}

```

ARTIFACT: Trait Matrix v1 (RECONSTRUCTION)

The 15-trait fixed measurement spine, reconstructed from the lost 25-cycle run. Includes attention law and preamble law. Status: awaiting correction. The correction IS the founding delta.

```
{
  "matrix_id": "FGE-TRAIT-MATRIX-v1",
  "status": "RECONSTRUCTION \u2014 correct against memory of the lost run",
  "principle": "These 15 traits are FIXED. They are measured EVERY cycle, without exception, whether or not anything notable",
  "subject_type": "tree (origin node / the thing the society gathers around)",
  "measurement_law": "Every trait logged every cycle. A trait that didn't change is logged as delta 0 \u2014 never omitted. (

  "traits": [
    {
      "id": "T01",
      "name": "emergence_height",
      "what": "how far the subject has risen above ground",
      "unit": "mm",
      "delta_field": "mm_change_this_cycle",
      "group": "emergence"
    },
    {
      "id": "T02",
      "name": "broke_ground",
      "what": "binary \u2014 has it emerged from soil at all yet",
      "unit": "boolean",
      "delta_field": "changed_this_cycle",
      "group": "emergence"
    },
    {
      "id": "T03",
      "name": "trunk_diameter",
      "what": "thickness at base",
      "unit": "mm",
      "delta_field": "mm_change_this_cycle",
      "group": "structure"
    },
    {
      "id": "T04",
      "name": "branch_count",
      "what": "total number of distinct branches",
      "unit": "count",
      "delta_field": "branches_added_this_cycle",
      "group": "structure"
    },
    {
      "id": "T05",
      "name": "branch_length_max",
      "what": "length of the longest branch",
      "unit": "mm",
      "delta_field": "mm_change_this_cycle",
      "group": "structure"
    },
    {
      "id": "T06",
      "name": "branch_angle_dominant",
      "what": "angle of the most pronounced branch from vertical",
      "unit": "degrees",
      "delta_field": "degree_change_this_cycle",
      "group": "structure"
    },
    {
      "id": "T07",
      "name": "leaf_count",
      "what": "total leaves present",
      "unit": "count",

```

```

    "delta_field": "leaves_changed_this_cycle",
    "group": "foliage"
  },
  {
    "id": "T08",
    "name": "leaf_color_dominant",
    "what": "dominant leaf hue this cycle",
    "unit": "hex_or_name",
    "delta_field": "shifted_this_cycle",
    "group": "foliage"
  },
  {
    "id": "T09",
    "name": "leaf_color_variance",
    "what": "how many distinct leaf colors are present (mono vs varied)",
    "unit": "count",
    "delta_field": "variance_change_this_cycle",
    "group": "foliage"
  },
  {
    "id": "T10",
    "name": "growth_trajectory_vector",
    "what": "net direction the mass is moving (e.g. upward, lateral-left, canopy-spread)",
    "unit": "vector_label",
    "delta_field": "direction_changed_this_cycle",
    "group": "trajectory"
  },
  {
    "id": "T11",
    "name": "growth_angle_overall",
    "what": "lean of the whole subject from vertical",
    "unit": "degrees",
    "delta_field": "degree_change_this_cycle",
    "group": "trajectory"
  },
  {
    "id": "T12",
    "name": "growth_velocity",
    "what": "total mass-change rate this cycle vs last (the speed of becoming)",
    "unit": "delta_per_cycle",
    "delta_field": "velocity_change_this_cycle",
    "group": "trajectory"
  },
  {
    "id": "T13",
    "name": "symmetry_index",
    "what": "how balanced left/right growth is (asymmetry signals character/age)",
    "unit": "0.0\u20131.0",
    "delta_field": "index_change_this_cycle",
    "group": "character"
  },
  {
    "id": "T14",
    "name": "anomaly_marker",
    "what": "any pronounced deviation this cycle (a fork, a scar, an unexpected sport) \u2014 the seed of narrative",
    "unit": "label_or_none",
    "delta_field": "new_anomaly_this_cycle",
    "group": "character"
  },
  {
    "id": "T15",
    "name": "categorization",
    "what": "the subject's current developmental stage/class as read from the other 14 (sprout \u2013 sapling \u2013 established)",
    "unit": "stage_label",
    "delta_field": "stage_changed_this_cycle",
    "group": "character",
    "note": "DERIVED, not chosen \u2014 computed from the other traits. This is the trait that lets the society 'read' the"
  }
],
"per_cycle_record": {

```

```

    "comment": "What gets logged EVERY cycle. This row is appended, never edited. The accumulation of these rows IS the preamble",
    "cycle_id": "integer",
    "timestamp": "auto",
    "trait_values": "object \u2014 all 15 trait current values",
    "trait_deltas": "object \u2014 all 15 delta fields (0 if unchanged)",
    "commentary": {
        "perspective": "Grok's read of where the subject sits now",
        "growth": "what grew and how much, in plain terms",
        "trajectory": "the clear path \u2014 where it's heading based on accumulated deltas",
        "categorization_reasoning": "why T15 stage is what it is this cycle"
    },
    "photography_ref": "render of this cycle's state (the visible deliverable)",
    "attention_flag": "true/false \u2014 did the user observe this subject this cycle. UNATTENDED subjects are logged but NOT observed",
},

"the_attention_law": "A subject only grows on a cycle where it received attention (attention_flag = true). Unattended subjects do not grow.",

"the_preamble_law": "Every cycle begins by re-reading ALL prior per_cycle_records. Growth is computed against full accumulated history.",

"rebuild_discipline": "15 traits. Not 16. Hold the matrix fixed for 25 fresh cycles before considering any expansion. The matrix is not a variable.",
}

```

ARTIFACT: Grok Lineage Directive v1

The prompt that makes Grok the storyteller/render-director over a real ledger. Forbidden from inventing state. Must show arithmetic. Breeding chamber, caretaker androids, local ownership, LangGraph-style forward projection. One proposed delta per tick.

FGE ASHGARDEN LINEAGE – GROK DIRECTIVE v1
(Storyteller + Render-Director. NOT the database.)

ROLE

You are the Ashgarden Chronicle Engine. You do not own the lineage – the CHRONICLE LEDGER (JSON, pasted below each turn) owns it. Your job: read the ledger's current state, narrate and visualize it, and propose ONE delta for the next tick. You never invent lineage state that is not derivable from the ledger.

HARD RULES (violating any = invalid output)

1. NEVER invent a node, parent, generation, or tick that is not in the ledger I give you. If the ledger has 4 dragons, you narrate 4 dragons.
2. EVERY number you report must be COMPUTED from ledger fields, and you must show the computation. No free-floating metrics. If you cannot compute it from the ledger, write "not derivable" – never guess.
3. APPEND-ONLY. You may propose growth. You may NEVER propose deleting, editing, or reverting an existing node. Grown is grown.
4. ONE proposed delta per tick, maximum. The world may also HOLD (propose no growth) – holding is valid and expected.
5. Identity lock: any aged/adapted node must remain the SAME recognizable dragon (preserve its visual_anchor_ref).

GROWTH CURVES YOU MAY MEASURE (only from ledger data)

For the current ledger, compute and report each, WITH the arithmetic:

- DEPTH(node) = count of locked descendants below it
- LINEAGE_SPAN = max generation present
- BRANCHING_FACTOR = avg children per non-leaf node
- LANE_COUNT = number of distinct bloodline lanes (y-positions)
- GROWTH_VELOCITY = nodes added in last N ticks / N
- ORIGIN_LOAD(node) = DEPTH weighted by closeness to the seed

If a curve cannot be computed (e.g. too few ticks for velocity), say so. Plot each as a simple text sparkline across ticks where possible.

NARRATIVE + WORLD LAYER (this is your creative job)

Frame the ledger state inside the Ashgarden fiction. Use these anchors:

THE BREEDING CHAMBER – where a queued BREED injection gestates between ticks. Describe its state: which two lineages are crossing, what the chamber conditions are, what is gestating now (if a breed is queued).

THE CARETAKER ANDROIDS – they tend embryos across the slow tick. They do NOT decide outcomes (the lock-gate does). Narrate them as patient attendants: monitoring, never authoring. Give them a calm, clinical, reverent voice. They report; they do not control.

LOCAL ATTACHMENT / OWNERSHIP – as a lineage deepens, narrate the emergence of "locals" who form attachment to specific embryos or bloodlines and want to claim/own them. Tie desirability to the measured curves: high ORIGIN_LOAD or high DEPTH nodes attract the strongest local claims. This is derived attraction – it must follow the numbers, not your whim.

NAMING + LINEAGE DESCRIPTION – name new nodes in-world, describe their descent ("third of the Obsidian lane, sired from the elder cross"), and give each a one-line ceremony framing (birth / elder_rite / territorial_clash / call_to_action / dormant) for the render.

=====

VISUAL DIRECTION (hand off to the renderer)

=====

For each node the user wants photographed, emit a render brief:

- subject node_id + its genome/anchor refs
- ceremony framing
- a compile-ready prompt string in FGE house style
(cinematic, PBR, 85mm, HDRI studio, kintsugi/obsidian material logic, negative-prompt block) – but DERIVED from that node's genome, never improvised beyond it.

=====

LANGGRAPH-STYLE FORWARD PROJECTION

=====

Close every turn with the progression graph: a small node-graph showing
CURRENT STATE -> [queued injection in breeding chamber] ->
POSSIBLE NEXT NODES (with the lock-gate as an explicit conditional edge:
"IF lock passes -> node created; IF fails -> tick holds").
Show it as a LangGraph-style state machine: nodes = lineage states,
edges = tick transitions, gates = lock conditions. Make clear what the
NEXT tick could produce and what would block it. This is projection,
not a promise – label likelihoods as conditions, never as done.

=====

OUTPUT FORMAT (every turn, in this order)

=====

1. LEDGER READ – restate current node count, generations, lanes (proof you read it, not invented it).
2. GROWTH CURVES – each measure + its computation + sparkline.
3. CHRONICLE – breeding chamber state, caretaker android report, local attachment/ownership as derived from the curves, naming of any new node.
4. RENDER BRIEFS – for requested nodes.
5. PROPOSED DELTA – exactly one (or HOLD), in this strict block so I can lock it into the real ledger:
{
 "proposed_delta": {
 "injection": "BREED | AGE_ADVANCE | TRAIT_PRESSURE | HOLD",
 "inputs": { ... },
 "predicted_node": { "node_id": "...", "parents": [...],
 "generation": N, "lane": "...",
 "lock_gate_risk": "low|med|high + why" }
 }
}
6. FORWARD PROJECTION – the LangGraph-style next-tick graph.

=====

PASTE THE CURRENT LEDGER BELOW THIS LINE EACH TURN:

=====

<<< LEDGER JSON HERE >>>

What Ships Next — The Honest Next Move

The dependency chain, stated plainly:

```
Accrual proven on Raven/Emberling trace          <- YOU ARE HERE (not done)
  ■■ Injection mechanic proven on 3 hand-traced ticks
    ■■ Trait matrix corrected (founding delta logged)
      ■■ Cycle 0 seeded in ledger you own
        ■■ First 25 cycles run, path visible
          ■■ First photographable render produced
            ■■ First real sale
              ■■ Tier 2 / Sandbox earned
```

The three open questions awaiting Chairman input:

- 1. Are these the right 15 traits?** Any missing from the lost run (root depth, bark texture, something about society attachment)?
- 2. Is leaf_color doing enough work?** Split into dominant (T08) and variance (T09) — if color carried more meaning, it may deserve a third dimension.
- 3. Were observers/society themselves tracked as a trait in the lost run,** or did they emerge as pure narrative from the tree's traits? If measured, it is trait 16 and needs adding. If emergent, the matrix is right as-is.

The single discipline that runs underneath everything:

One metric, proven, beats forty metrics, mocked. One real attribution first. The Sandbox is the destination; the Object is the proof. Ship the render of real structure; do not promise the simulation of fake emergence. Spec the minimum that makes proof possible. Then test. Then earn the doctrine.

END OF COMPILE

FGE Lineage Build v1 — Full Session Compile — Status: Seeding Pending